



COMPLEX AUTOMATION PRO

# CAP

Мониторинг обогатительной фабрики  
в реальном времени — от приёма руды до отгрузки концентрата

MVP · сентябрь 2026

[github.com/4codegit/complex\\_automation\\_pro](https://github.com/4codegit/complex_automation_pro) · лицензия MIT



## Задача: завод видит только часть себя

Технологические участки фабрики разрознены: сигналы живут в локальных контроллерах, оператор узнаёт о проблеме по звонку, а обрыв связи означает потерю данных.

- ▶ Данные с датчиков не доходят до оператора
- ▶ История измерений не хранится централизованно
- ▶ Отклонения от регламента видны постфактум

9

технологических участков  
под наблюдением

3

промышленных протокола:  
Modbus, OPC UA, Sparkplug B

0

команд записи в АСУ ТП —  
платформа только наблюдает



# Архитектура: edge-шлюзы и микросервисы



Все сервисы — отдельные процессы из одного кода; в режиме разработки работают как один процесс. Контракт API и панель не меняются между режимами.



# Три промышленных протокола из коробки

## Modbus TCP

Только чтение, функции 1–4. Регистры: bool, u16...f32, масштаб, перестановка слов CDAB. Реальный источник для демо — планшет с Modbus-сервером.

## OPC UA

Poll и подписки (subscriptions), безопасность Sign/SignAndEncrypt, аутентификация по сертификату x509, анонимный или логин/пароль.

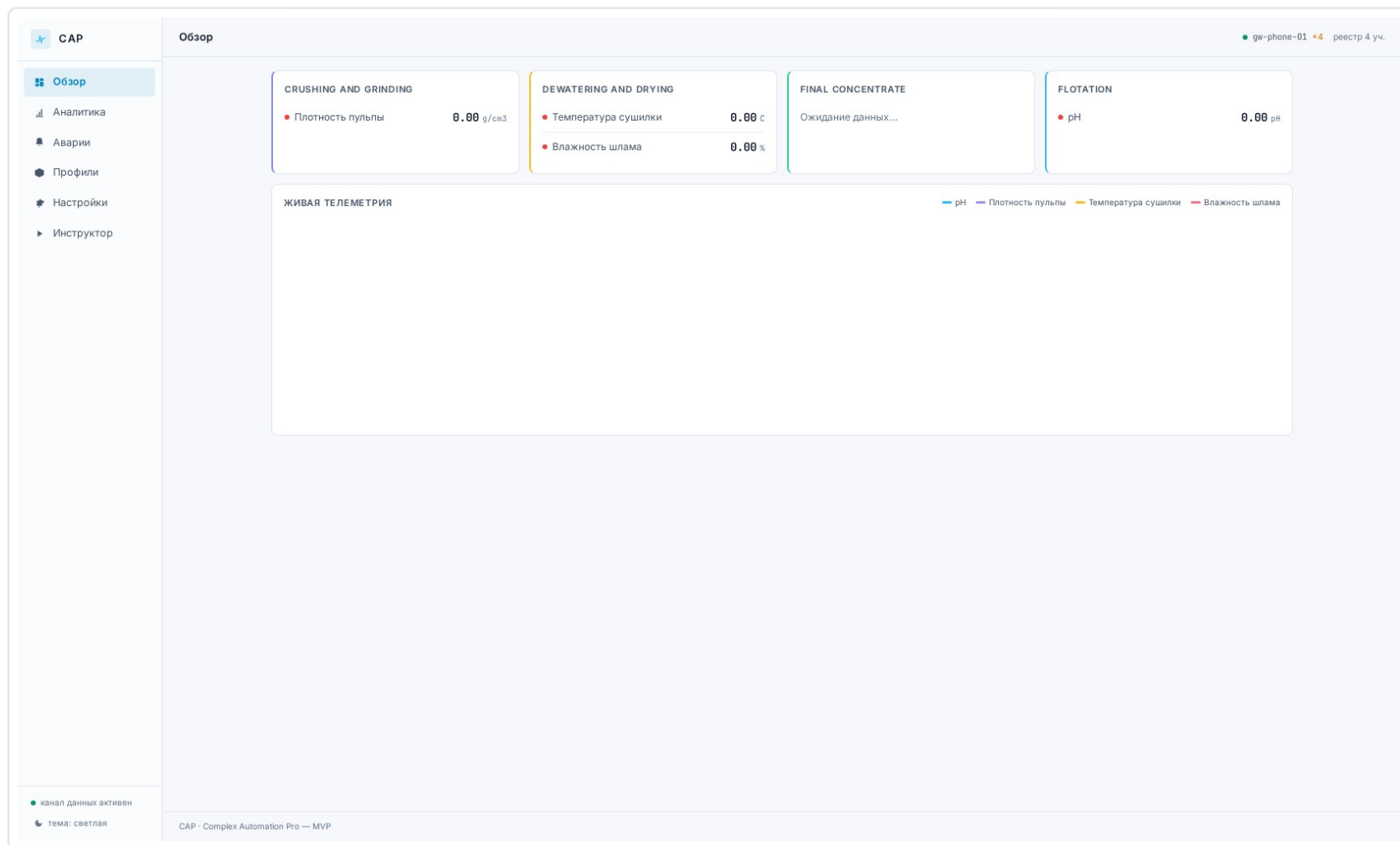
## MQTT Sparkplug B

Роль Primary Host Application: подписка spBv1.0/#, разрешение alias → name по Birth-сообщениям, protobuf-декодер без кодогенерации.

**Граница read-only — в коде:** драйверы не содержат функций записи (FC 5/6/15/16 отсутствуют). Конфигурация сигналов — одна строка на тег: `mill_power:kW|reg=hr:100:f32`



# Панель оператора



## Участки и живой график

карточки стадий, WebSocket-поток

## Качество каждого сигнала

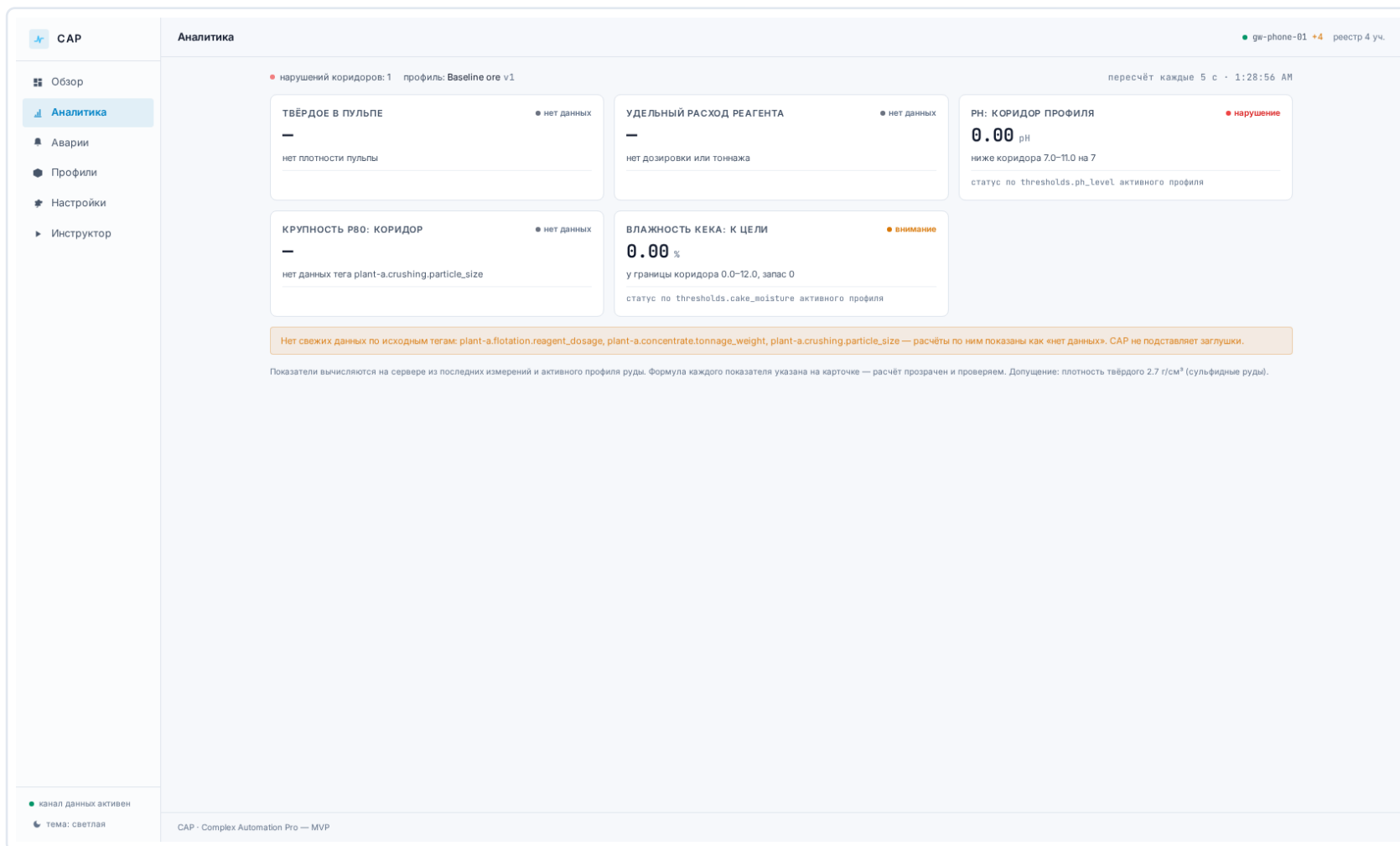
good / устаревшее / нет связи

## Светлая и тёмная темы

переключатель в меню



# Металлургия считается, а не угадывается



## Твёрдое в пульпе

$$\% S = \frac{\rho_t (\rho_p - \rho_v)}{(\rho_t - \rho_v) \rho_p}$$

## Удельный расход

$$q = Q_p \cdot 60 / Q_t \text{ [мл/т]}$$

## Сухая производительность

$$Q_{\text{сух}} = Q_t \cdot (1 - W / 100)$$

Формула на каждой карточке; нет данных — честное «нет данных», без заглушек.

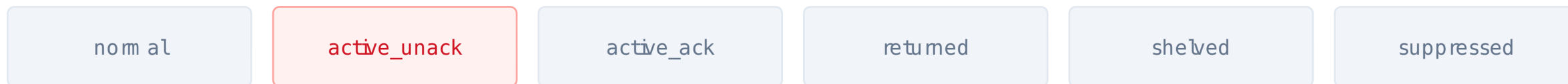
# Обрыв связи не теряет ни одного измерения



64

сообщений пережили демонстрационный отказ сервера — и все 64 дошли после восстановления, без потерь и дублей (сценарий demo.sh)

# Аварии по ISA-18.2, каждое действие — в аудите



Жизненный цикл аварии управляется уставками активного профиля руды.

## Квитирование

оператор подтверждает аварию; действие фиксируется в неизменяемом журнале аудита: кто, что, когда

## Рационализация

уставки по каждому тегу согласуются технологом и версионируются через профили

## Права доступа

RBAC: manage\_tags, acknowledge\_alarms, manage\_profiles... субъект без роли получает 403 и запись в аудите





## Наблюдает — но не управляет

# Потеря или компрометация CAP не останавливает производство.

### Уровень 3 по ISA-95

чтение из уровней 0–2; контуры регулирования, блокировки и аварийные остановки остаются в независимых системах

### Только чтение

в коде драйверов нет операций записи; pull используется для дозаполнения истории, никогда — для команд

### Прозрачность данных

коды качества good/uncertain/bad/offline: плохие данные не превращаются в нормальные, прорехи видны



# Статус MVP и дорожная карта

## Готово и проверено

- ▶ 7 микросервисов + edge-шлюз, живой прогон
- ▶ Драйверы Modbus TCP, OPC UA, Sparkplug B
- ▶ Store-and-forward: отказ → полный backfill
- ▶ Расчётные показатели с формулами
- ▶ Аварии ISA-18.2, RBAC, аудит, профили руды
- ▶ Контрактные тесты, светлая и тёмная темы

## Дальше

- ▶ Полевые испытания драйверов на оборудовании площадки
- ▶ Брокер событий (NATS) вместо точечных EVENT\_SINKS
- ▶ Перенос ядра на Rust, панель оператора — на Slint
- ▶ PostgreSQL-профиль, резервирование, приёмочные испытания



Комплексная автоматизация, которую можно проверить:  
открытый код, прозрачные формулы, честные прорехи в данных.

[github.com/4codegit/complex\\_automation\\_pro](https://github.com/4codegit/complex_automation_pro)  
Лицензия MIT · Правообладатель 4codegit · 2026